토큰화 과정 (Tokenization Process)

개요

이 문서는 Monkey 프로그래밍 언어의 토큰화 과정을 상세히 설명합니다. 토큰화는 소스 코드를 의미 있는 토큰들로 분해하는 첫 번째 단계입니다.

목차

1. Lexer 구조체와 토큰화 개념
2. NextToken() 함수 동작 방식
3. TokenType 상수와 Token 구조체

---

1. Lexer 구조체와 토큰화 개념

1.1 토큰화의 기본 원리와 필요성

토큰화(Tokenization)는 컴파일러나 인터프리터의 첫 번째 단계로, 소스 코드 문자열을 의미 있는 단위인 **토큰(Token)**으로 분해하는 과정입니다.

토큰화가 필요한 이유:

1. **구문 분석 단순화**: 문자 단위가 아닌 의미 단위로 처리하여 파서의 복잡성을 줄입니다
2. **에러 처리 개선**: 잘못된 문자나 구문을 조기에 발견할 수 있습니다
3. **성능 최적화**: 한 번의 스캔으로 모든 토큰을 식별하여 효율성을 높입니다
4. **언어 구조 명확화**: 키워드, 연산자, 식별자 등을 명확히 구분합니다

토큰화 과정 예시:

```
입력: "let x = 5 + 10;"
```

```
토큰화 결과:
1. {Type: "Let", Literal: "let"}
2. {Type: "Identifier", Literal: "x"}
3. {Type: "=", Literal: "="}
4. {Type: "Int", Literal: "5"}
5. {Type: "+", Literal: "+"}
6. {Type: "Int", Literal: "10"}
7. {Type: ";", Literal: ";"}
```

1.2 Lexer 구조체의 구조와 각 필드 역할

Monkey 언어의 `Lexer` 구조체는 다음과 같이 정의됩니다:

```go
type Lexer struct {
input  string  // 토큰화할 입력 문자열
position  int  // 현재 문자의 위치 (현재 읽고 있는 문자)
nextPosition  int  // 다음 문자의 위치 (다음에 읽을 문자)
ch  byte  // 현재 검사 중인 문자
}
```

각 필드의 상세 역할:

1. **`input` 필드:**
   - 토큰화할 전체 소스 코드 문자열을 저장
   - 한 번 설정되면 변경되지 않는 읽기 전용 데이터
   - 예: `"let x = 5;"`
2. **`position` 필드:**
   - 현재 읽고 있는 문자의 인덱스 위치
   - `ch` 필드에 저장된 문자의 `input` 내 위치를 나타냄
   - 토큰의 시작과 끝 위치를 계산할 때 사용
3. **`nextPosition` 필드:**
   - 다음에 읽을 문자의 인덱스 위치
   - `position + 1` 과 같지만, 미리 계산해두어 성능을 향상
   - `peekChar()` 함수에서 다음 문자를 미리 확인할 때 사용
4. **`ch` 필드:**
   - 현재 검사 중인 문자를 저장
   - `byte` 타입으로 ASCII 문자를 처리
   - 0 값은 EOF(End of File)를 의미

Lexer 초기화 과정:

```
func New(input string) *Lexer {
l := &Lexer{input: input}
l.readChar() // 첫 번째 문자를 읽어서 ch에 설정
return l
```

```
}
```

초기화 시 `readChar()` 를 호출하여 첫 번째 문자를 `ch` 에 로드하고, `position` 과 `nextPosition` 을 적절히 설정합니다.

1.3 토큰화 과정의 전체 흐름

1. **초기화**: `New()` 함수로 Lexer 생성 및 첫 문자 로드
2. **토큰 추출**: `NextToken()` 함수를 반복 호출하여 토큰 하나씩 추출
3. **문자 진행**: `readChar()` 로 다음 문자로 이동
4. **종료**: EOF 토큰을 만날 때까지 반복

이러한 구조를 통해 Lexer는 효율적으로 소스 코드를 스캔하면서 각 문자를 적절한 토큰으로 변환할 수 있습니다.

2. N

extToken() 함수 동작 방식

2.1 NextToken() 함수의 전체 구조

`NextToken()` 함수는 Lexer의 핵심 함수로, 현재 위치에서 하나의 토큰을 추출하여 반환합니다.

```
func (l *Lexer) NextToken() token.Token {
var  tok token.Token
l.skipWhitespace() // 공백 문자 건너뛰기
switch  l.ch {
case  '=':
// 할당 연산자 또는 동등 비교 연산자 처리
```

```
case '+':
// 덧셈 연산자 처리
// ... 기타 문자들
default:
// 식별자, 숫자, 또는 잘못된 문자 처리
}
l.readChar() // 다음 문자로 이동
return tok
}
```

2.2 Switch 문을 통한 문자별 토큰 처리

2.2.1 단일 문자 토큰 처리

대부분의 연산자와 구분자는 단일 문자로 구성되어 간단하게 처리됩니다:

```
case '+':
tok = newToken(token.Plus, l.ch)
case '-':
tok = newToken(token.Minus, l.ch)
case '*':
tok = newToken(token.Asterisk, l.ch)
case '/':
tok = newToken(token.Slash, l.ch)
case '(':
```

```
tok = newToken(token.LeftParen, l.ch)
case ')':
tok = newToken(token.RightParen, l.ch)
```

`newToken()` 헬퍼 함수는 토큰 타입과 현재 문자를 받아 Token 구조체를 생성합니다:

```
func newToken(tokenType token.TokenType, ch byte) token.Token {
return token.Token{Type: tokenType, Literal: string(ch)}
}
```

2.2.2 다중 문자 토큰 처리

일부 연산자는 두 문자로 구성되어 `peekChar()` 를 사용해 다음 문자를 확인합니다:

```
case '=':
if l.peekChar() == '=' {
ch := l.ch
l.readChar() // '=' 문자로 이동
literal := string(ch) + string(l.ch) // "=="
tok = token.Token{Type: token.Equal, Literal: literal}
} else {
tok = newToken(token.Assign, l.ch) // "="
}

case '!':
```

```go
if l.peekChar() == '=' {
ch := l.ch
l.readChar() // '=' 문자로 이동
literal := string(ch) + string(l.ch) // "!="
tok = token.Token{Type: token.NotEqual, Literal: literal}
} else {
tok = newToken(token.Bang, l.ch) // "!"
}
```

2.2.3 문자열 리터럴 처리

문자열은 따옴표로 둘러싸인 여러 문자로 구성됩니다:

```go
case '"':
tok.Type = token.String
tok.Literal = l.readString()
```

`readString()` 함수는 닫는 따옴표까지 모든 문자를 읽어 문자열을 구성합니다:

```go
func (l *Lexer) readString() string {
pos := l.position + 1 // 시작 따옴표 다음 위치
for {
l.readChar()
if l.ch == '"' || l.ch == 0 { // 닫는 따옴표 또는 EOF
break
```

```
}
}
return l.input[pos:l.position] // 따옴표 사이의 내용 반환
}
```

2.2.4 Default 케이스: 식별자, 숫자, 잘못된 문자

Switch 문에서 처리되지 않는 문자들은 default 케이스에서 처리됩니다:

```
default:
if isLetter(l.ch) {
tok.Literal = l.readIdentifier()
tok.Type = token.LookupIdentifierType(tok.Literal)
return tok // readChar() 호출하지 않고 바로 반환
} else if isDigit(l.ch) {
tok.Literal = l.readNumber()
tok.Type = token.Int
return tok // readChar() 호출하지 않고 바로 반환
} else {
tok = newToken(token.Illegal, l.ch)
}
```

2.3 readChar()와 peekChar() 함수의 역할

2.3.1 readChar() 함수

현재 문자를 다음 문자로 진행시키는 핵심 함수입니다:

```
func (l *Lexer) readChar() {
l.ch = l.peekChar() // 다음 문자를 현재 문자로 설정
l.position = l.nextPosition  // 위치 업데이트
l.nextPosition += 1  // 다음 위치 증가
}
```

동작 과정:

1. `peekChar()` 로 다음 문자를 가져와 `ch` 에 설정
2. `position` 을 `nextPosition` 으로 업데이트
3. `nextPosition` 을 1 증가시켜 그 다음 문자를 가리키도록 함

2.3.2 peekChar() 함수

다음 문자를 미리 확인하되 위치는 변경하지 않는 함수입니다:

```
func (l *Lexer) peekChar() byte {
if  l.nextPosition >= len(l.input) {
return  0  // EOF
} else {
return  l.input[l.nextPosition]
}
}
```

사용 목적:

- 다중 문자 토큰 (`==` , `!=`) 식별
- 현재 위치를 변경하지 않고 다음 문자 확인
- EOF 검사 (입력 끝 도달 시 0 반환)

2.4 공백 문자 처리

토큰화 전에 의미 없는 공백 문자들을 건너뛰는 과정입니다:

```
func (l *Lexer) skipWhitespace() {
for  l.ch == ' ' || l.ch == '\t' || l.ch == '\n' || l.ch == '\r' {
l.readChar()
}
}
```

이 함수는 공백, 탭, 개행 문자를 만날 때마다 `readChar()` 를 호출하여 의미 있는 문자가 나올 때까지 건너뜁니다.

2.5 NextToken() 함수 실행 예시

입력: `"x = 5"`

1. **첫 번째 호출**: `x`

- `skipWhitespace()` 실행 (공백 없음)
- `l.ch = 'x'` → default 케이스
- `isLetter('x') = true` → `readIdentifier()` 호출
- 반환: `{Type: "Identifier", Literal: "x"}`

2. **두 번째 호출**: `=`

- `skipWhitespace()` 실행 (공백 건너뛰기)

- `l.ch = '='` → case '='
- `peekChar() = ' '` (공백이므로 단일 '=')
- 반환: `{Type: "=", Literal: "="}`

3. **세 번째 호출**: `5`

- `skipWhitespace()` 실행 (공백 건너뛰기)
- `l.ch = '5'` → default 케이스
- `isDigit('5') = true` → `readNumber()` 호출
- 반환: `{Type: "Int", Literal: "5"}` ## 3

. TokenType 상수와 Token 구조체

3.1 Token 구조체의 구조와 필드 역할

Monkey 언어의 `Token` 구조체는 매우 간단하지만 효과적인 설계를 가지고 있습니다:

```
type Token struct {
Type TokenType // 토큰의 종류를 나타내는 타입
Literal  string  // 토큰의 실제 문자열 값
}
```

각 필드의 역할:

1. `Type` 필드 (TokenType):

- 토큰의 종류를 식별하는 열거형 상수
- 파서가 토큰의 의미를 이해하는 데 사용
- 예: `token.Let`, `token.Identifier`, `token.Plus`

2. `Literal` 필드 (string):

- 소스 코드에서 실제로 나타난 문자열 값
- 식별자의 이름, 숫자의 값, 연산자의 기호 등을 저장
- 예: `"let"`, `"myVariable"`, `"123"`, `"+"`

3.2 TokenType 상수 분류

`TokenType` 은 문자열 타입의 별칭으로 정의되며, 다양한 토큰 종류를 상수로 정의합니다:

```
type TokenType string
```

3.2.1 특수 토큰

```
const (
Illegal = "Illegal"  // 인식할 수 없는 문자
EOF = "EOF"  // 파일 끝
)
```

- **Illegal**: 렉서가 인식할 수 없는 문자를 만났을 때 사용
- **EOF**: 입력의 끝에 도달했음을 나타내는 특별한 토큰

3.2.2 식별자와 리터럴

```
const (
Identifier = "Identifier"  // 변수명, 함수명 등
Int = "Int"  // 정수 리터럴
String = "String"  // 문자열 리터럴
```

```
)
```

- **Identifier**: 사용자가 정의한 변수명, 함수명 등
- **Int**: 숫자 리터럴 (예: `123`, `456`)
- **String**: 문자열 리터럴 (예: `"hello"`, `"world"`)

3.2.3 연산자 (Operators)

```go
const (
Assign = "="  // 할당 연산자
Plus = "+"  // 덧셈
Minus = "-"  // 뺄셈
Bang = "!"  // 논리 부정
Asterisk = "*"  // 곱셈
Slash = "/"  // 나눗셈
Equal = "=="  // 동등 비교
NotEqual = "!="  // 부등 비교
LessThan = "<"  // 미만
GreaterThan = ">"  // 초과
)
```

연산자 분류:

- **산술 연산자**: `+`, `-`, `*`, `/`
- **비교 연산자**: `==`, `!=`, `<`, `>`
- **할당 연산자**: `=`

- **논리 연산자**: !

3.2.4 구분자 (Delimiters)

```
const (
Comma = ","  // 쉼표
Semicolon = ";"  // 세미콜론
Colon = ":"  // 콜론
LeftParen = "("  // 왼쪽 소괄호
RightParen = ")"  // 오른쪽 소괄호
LeftBrace = "{"  // 왼쪽 중괄호
RightBrace = "}"  // 오른쪽 중괄호
LeftBracket = "["  // 왼쪽 대괄호
RightBracket = "]"  // 오른쪽 대괄호
)
```

구분자 용도:

- **괄호류**: 함수 호출, 블록, 배열 등의 경계 표시
- **구분 기호**: 매개변수, 문장 등의 구분

3.2.5 키워드 (Keywords)

```
const (
Function = "Function"  // fn
Let = "Let"  // let
True = "True"  // true
```

```
False = "False"  // false

If = "If"  // if

Else = "Else"  // else

Return = "Return"  // return

)
```

키워드는 언어에서 특별한 의미를 가지는 예약어들입니다.

3.3 키워드 식별 시스템

3.3.1 keywords 맵

키워드와 일반 식별자를 구분하기 위해 맵을 사용합니다:

```
var  keywords = map[string]TokenType{

"fn": Function,

"let": Let,

"true": True,

"false": False,

"if": If,

"else": Else,

"return": Return,

}
```

이 맵은 소스 코드의 실제 키워드 문자열을 해당하는 TokenType 상수로 매핑합니다.

3.3.2 LookupIdentifierType() 함수

식별자가 키워드인지 일반 식별자인지 판단하는 함수입니다:

```
func LookupIdentifierType(identifier string) TokenType {
if tok, ok := keywords[identifier]; ok {
return tok // 키워드인 경우 해당 토큰 타입 반환
}
return Identifier // 일반 식별자인 경우 Identifier 반환
}
```

동작 과정:

1. `keywords` 맵에서 입력된 문자열을 검색
2. 키워드가 발견되면 해당 TokenType 반환
3. 키워드가 아니면 `Identifier` 반환

3.3.3 키워드 식별 예시

```
// 키워드 예시
LookupIdentifierType("let") // → token.Let
LookupIdentifierType("if") // → token.If
LookupIdentifierType("true") // → token.True

// 일반 식별자 예시
LookupIdentifierType("myVar") // → token.Identifier
LookupIdentifierType("count") // → token.Identifier
```

```
LookupIdentifierType("x") // → token.Identifier
```

3.4 토큰 생성과 사용 예시

3.4.1 다양한 토큰 생성 예시

```
// 키워드 토큰
letToken := token.Token{Type: token.Let, Literal: "let"}

// 식별자 토큰
identToken := token.Token{Type: token.Identifier, Literal: "myVariable"}

// 연산자 토큰
plusToken := token.Token{Type: token.Plus, Literal: "+"}

// 리터럴 토큰
intToken := token.Token{Type: token.Int, Literal: "42"}
stringToken := token.Token{Type: token.String, Literal: "hello"}
```

3.4.2 실제 토큰화 결과 예시

입력 코드: `let result = add(5, 3);`

토큰화 결과:

```
[
{Type: "Let", Literal: "let"},
```

```
{Type: "Identifier", Literal: "result"},
{Type: "=", Literal: "="},
{Type: "Identifier", Literal: "add"},
{Type: "(", Literal: "("},
{Type: "Int", Literal: "5"},
{Type: ",", Literal: ","},
{Type: "Int", Literal: "3"},
{Type: ")", Literal: ")"},
{Type: ";", Literal: ";"},
{Type: "EOF", Literal: ""}
]
```

이러한 토큰 시스템을 통해 Monkey 언어는 소스 코드를 구조화된 토큰 스트림으로 변환하여, 다음 단계인 구문 분석(Parsing)에서 효율적으로 처리할 수 있게 됩니다.

요약

토큰화 과정은 다음과 같은 핵심 요소들로 구성됩니다:

1. **Lexer 구조체**: 입력 문자열을 순차적으로 스캔하며 토큰을 추출
2. **NextToken() 함수**: Switch 문을 통해 각 문자를 적절한 토큰으로 변환
3. **Token 구조체**: 토큰의 타입과 실제 값을 저장하는 간단한 데이터 구조
4. **TokenType 상수**: 다양한 토큰 종류를 체계적으로 분류
5. **키워드 시스템**: 예약어와 일반 식별자를 구분하는 메커니즘

이러한 토큰화 시스템을 통해 Monkey 언어는 소스 코드를 의미 있는 단위로 분해하여 다음 단계인 구문 분석을 위한 기반을 마련합니다.